“File Permission Auditor”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

By

P NEEHARIKA	**2520090233**
G SHIVANI	**2520090046**

Under the Esteemed Guidance of

Raghupathi M

Department of Computer Science & Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"File Permission Auditor"** is a record of Bonafide work of **P NEEHARIKA - 2520090233, G SHIVANI - 2520090046** submitted in partial fulfillment for the award of B. Tech in Computer Science & Information Technology to the K L (Deemed to be) University. The results embodied in this report have not been copied from any other departments/University/Institute.

P NEEHARIKA	**2520090233**
G SHIVANI	**2520090046**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **"File Permission Auditor"** is a bonafide work done and submitted by **P Neeharika – 2520090233, G Shivani – 2520090046** for the course Operating Systems and System Programming (25CS2104E),

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science & Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our Course Coordinator and to our guide Raghupathi M, Department of Computer Science & Information Technology, which cannot be expressed in words, for the tremendous support, encouragement and guidance given for this project.

We express our gratitude to **Khaja Shareef (CSIT)**, Head of the ***Department for Computer Science & Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P NEEHARIKA 2520090233

G SHIVANI 2520090046

Abstract

Operating systems protect data by controlling who may read, write or execute every file, yet incorrectly configured permissions remain one of the most common causes of accidental modification and unauthorized access on Linux systems. Checking the permissions of a large number of files by hand is slow and error-prone. This project, the File Permission Auditor, is a security-focused Linux application written in C that automates this task for a user-selected directory.

The application scans the chosen directory and all of its sub-directories recursively and examines the read, write and execute permissions of the owner, the group and other users for every file. Using a set of rules, each file is classified as Safe, Warning or Critical depending on the access it grants, for example write access given to other users or the presence of a SetUID bit. The detected issues are displayed in a clear tabular format. The user can then correct the unsafe permissions of selected files, and every change is recorded in an Audit Log containing the file name, the old permission, the new permission and the time of change. A Permission Report summarises the total number of files scanned and the number of Safe, Warning and Critical files.

The project is implemented with the C language, the GCC compiler and Linux system calls such as stat, lstat, opendir, readdir and chmod. It demonstrates important Operating System and System Programming concepts including file systems, directory traversal, file permissions, ownership, access control, system calls and file handling, and gives practical insight into how an operating system protects files and controls access to them.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10–11
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		12
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		13
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		14–16
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**17–19**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**20–24**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**25**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**26**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER – 1

INTRODUCTION

1.1 Background of the Project

Every file on a Linux system has an owner, a group and a set of permissions that say who can read it, write to it or run it. These permissions are the first line of protection the operating system gives to our data. The problem is that they are easy to get wrong. A quick chmod 777 to make an error go away, a script copied from another machine, or an installer with sloppy defaults can leave files open to everybody.

The File Permission Auditor is a command-line tool written in C for Linux. It goes through a directory, looks at the permissions of every file and folder in it and tells the user which ones look unsafe.

1.2 Problem Statement

Checking permissions by hand with ls -l is slow when a directory has hundreds of files, and the permission strings are easy to misread. Nothing in the normal commands tells you which files are actually risky, and chmod -R changes everything without leaving any record. This project scans a directory and its sub-directories, marks each file as Safe, Warning or Critical, lets the user fix the unsafe ones and writes every change to an audit log.

1.3 Scope of the Project

- Select a directory and scan it recursively.
- Check read, write and execute permissions for owner, group and others.
- Check the SetUID and SetGID bits.
- Classify every file as Safe, Warning or Critical.
- Fix one selected file or all unsafe files.
- Keep an audit log of the changes.
- Show a report with the number of files in each category.

ACLs, changing the owner with chown and a graphical interface are not part of this version.

1.4 Objectives of the Project

- Develop a Linux-based application for checking file and directory permissions.
- Scan directories and sub-directories recursively.
- Analyze permissions of the owner, group and other users.
- Identify unsafe permissions and classify files as Safe, Warning or Critical.
- Provide an option to correct unsafe permissions.
- Maintain an audit log with file name, old permission, new permission and time of change.
- Generate a permission report.
- Understand Linux file systems, ownership, access control and system calls in practice.

1.5 Importance of the Application

Wrong permissions are a common and quiet security problem. A file that anybody can write to can be changed by accident or on purpose, and nobody notices until something breaks. The tool finds such files in seconds and shows exactly what was changed afterwards. It also turns the theory of file permissions from the OS course into something we can run and see.

1.6 Target Users / Audience

System administrators, developers who want to check a project folder before deploying it, and students learning Linux file permissions and system calls.

CHAPTER – 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Concept	Application in the Project
File System	Files and directories are stored with an inode that holds the owner, group and permission bits.
File Permissions	The read, write and execute bits for owner, group and others are checked for every file.
Ownership	Every file belongs to one user and one group; the kernel uses this to decide which permission set applies.
Access Control	Linux uses discretionary access control, so the owner decides who else can access a file. Extra access beyond what is needed is what the tool looks for.
Directory Traversal	A recursive function visits every sub-directory under the selected one.
System Calls	stat/lstat, opendir/readdir and chmod are used to read and change file information.
Special Bits	SetUID, SetGID and the sticky bit change how a file or directory behaves and are treated as risky.
File Handling	The audit log is created and appended using the standard file I/O functions.

A Linux permission mode is stored as three groups of bits. For example 644 means the owner can read and write, and the group and others can only read.

Class	Read (4)	Write (2)	Execute (1)
Owner	S_IRUSR	S_IWUSR	S_IXUSR
Group	S_IRGRP	S_IWGRP	S_IXGRP
Others	S_IROTH	S_IWOTH	S_IXOTH

2.2 Existing Methods / Algorithms

The usual way to check permissions is ls -l, stat or find -perm. They work, but the output is long, every line has to be read and understood by the user, and there is no summary. chmod -R can fix permissions in bulk, but it changes safe files as well and does not record what it did. Full security scanners exist, but they are large tools and are not meant for a quick check of one folder.

2.3 Proposed Approach

1. The user selects a directory.
2. The directory and its sub-directories are scanned recursively.
3. The permissions of each file are read with lstat().
4. Each file is classified using fixed rules (shown below).
5. The user can correct the unsafe files with chmod().
6. Every change is written to the audit log.
7. A report with the totals is displayed.

The rules used for classification:

Level	Condition
Critical	SetUID bit is set, or SetGID bit on a file, or the file is writable by other users
Warning	The file is writable by the group, or a regular file is executable by other users
Safe	None of the above

A directory with the sticky bit set (like /tmp) is allowed to be writable by others, so it is not marked Critical for that reason.

2.4 Advantages of the Proposed Approach

- One scan covers the whole directory tree.
- The result is easy to read: Safe, Warning or Critical with a reason.
- Only the unsafe bits are removed when fixing, so other permissions stay as they were.
- Every change is recorded in the audit log.
- Written in plain C, no extra libraries are needed.
- Easy to test on Ubuntu/WSL.

CHAPTER – 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Requirement	Specification
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	10 GB free space

3.2 Software Requirements

Requirement	Specification
Operating System	Ubuntu / Linux
Programming Language	C
Compiler	GCC
Text Editor	Nano / VS Code
Version Control	Git / GitHub

3.3 Operating System Environment – Ubuntu / WSL

The program uses Linux file permissions and POSIX calls, so it has to run on Linux. On a Windows laptop it can be run in Ubuntu through WSL. The folder that is scanned should be inside the Linux file system (for example under /home), because folders on the Windows drives under /mnt do not allow chmod to change permissions properly.

3.4 Development Tools and Compilers

The program is a single C file compiled with GCC. Nano or VS Code was used for editing, the terminal for testing, and Git/GitHub to keep the source code.

CHAPTER – 4

TECHNOLOGY AND TOOLS USED

Technology / Tool	Purpose
C Programming	Implementation of the scanner, risk analysis, correction and log.
GCC Compiler	Compiles file_permission_auditor.c into the executable.
Linux System Calls	stat, lstat, opendir, readdir, closedir and chmod.
C Standard Library	printf, scanf, fopen, malloc, realloc, snprintf, time functions.
Linux Commands	ls -l, stat, chmod, mkdir, touch, cat, find.
Git / GitHub	Version control and project storage.

4.1 C / C++ Programming

C gives direct access to the system-call interface and to structures like struct stat, which is exactly the information we need. Only C is used in this project.

4.2 GCC Compiler

The program is compiled with gcc -Wall -Wextra -o auditor file_permission_auditor.c. No extra libraries have to be linked.

4.3 Linux System Calls

Function	Purpose
stat()	Checks that the path entered by the user is a directory.
lstat()	Reads the mode, owner and group of each entry without following symbolic links.
opendir() / readdir() / closedir()	Reads the names inside a directory.
chmod()	Applies the corrected permissions.
time() / localtime() / strftime()	Time stamp for the audit log.

4.4 POSIX Threads / Synchronization

Threads are not used. The program is a single process with a single thread, because the scan is a simple one-after-another walk through the folders and the program keeps one shared list and a few counters. Scanning with several threads could be added later.

4.5 Linux Commands and Utilities

ls -l and stat to look at permissions, chmod to create test files with different permissions, mkdir and touch to build the test folder, find -perm to cross-check the results and cat to read the audit log.

4.6 Other Tools / Libraries Used

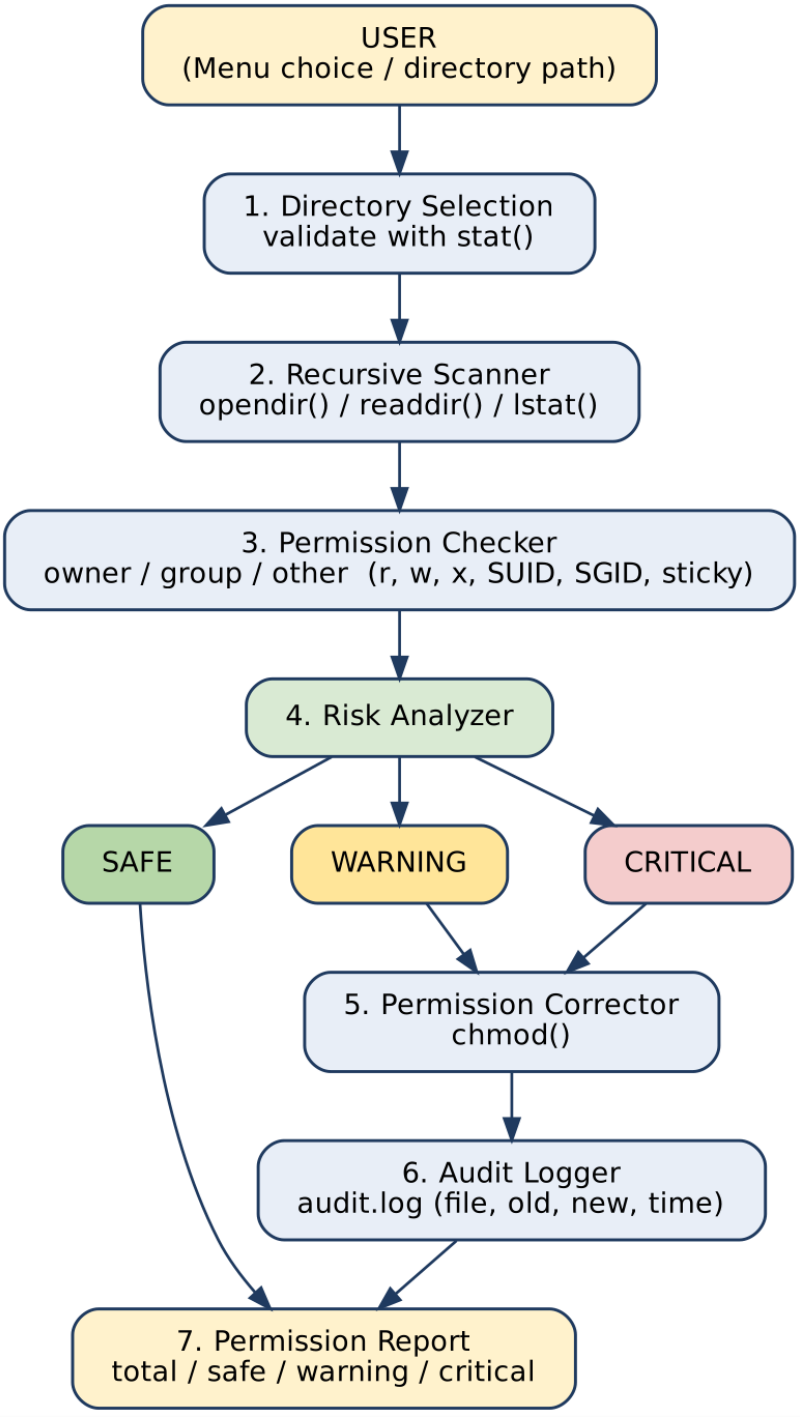
Git and GitHub for the source code, and gdb and valgrind for debugging and checking memory use.

CHAPTER – 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application works as a pipeline. The user gives a directory, the scanner walks through it, each file is checked and classified, and the user can then fix the unsafe files. All fixes go to the audit log and the totals are shown in the report.

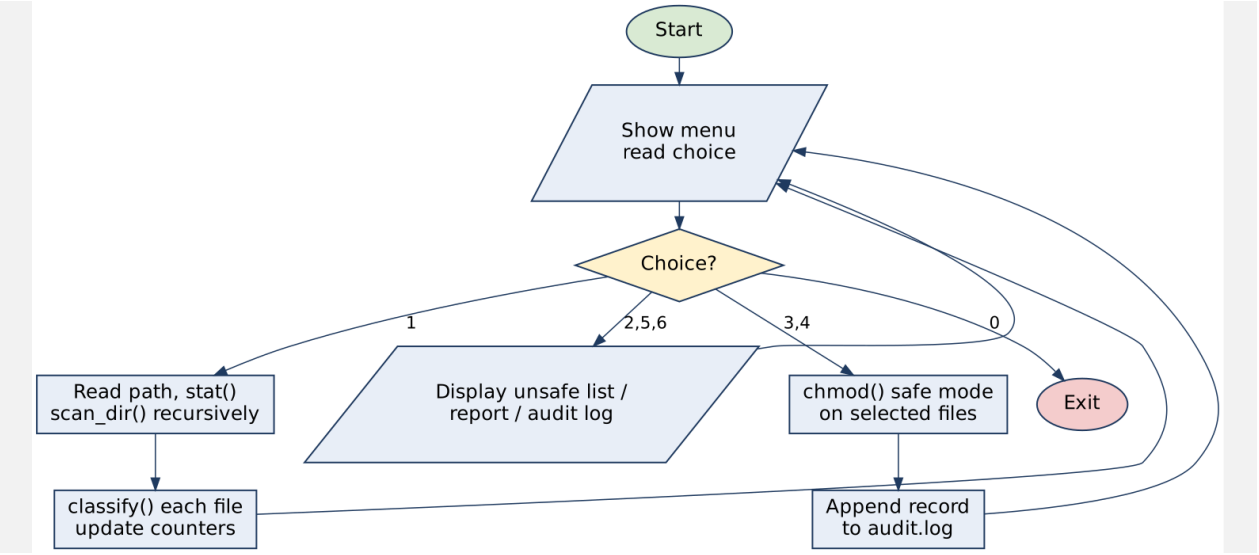


5.2 System Workflow

1. User Input – select the directory to be scanned.

2. Recursive Scanning – scan the directory and its sub-directories.
3. Permission Checking – check read, write and execute for owner, group and others.
4. Risk Analysis – classify each file as Safe, Warning or Critical.
5. Permission Correction – remove the unsafe bits of the selected files.
6. Audit Logging – record file name, old permission, new permission and time.
7. Permission Report – show total files and the Safe, Warning and Critical counts.

5.3 Flowchart / Data Flow



5.4 Module Description

Module	Description
Directory Selection	Reads the path and checks it with stat().
Recursive Scanner	scan_dir() reads each directory and calls itself for sub-directories.
Permission Checker	Stores the mode, owner and group of every file.
Risk Analyzer	classify() returns Safe, Warning or Critical with a reason.
Permission Corrector	Removes the unsafe bits using chmod().
Audit Logger	Writes each change to audit.log.
Report Generator	Shows the unsafe files and the summary counts.

5.5 Input and Output Design

Input is a menu choice (0 to 6), the directory path and, for fixing a single file, the ID shown in the list of unsafe files. Output is the list of unsafe files, the permission report, the messages for each fix and the audit log.

```
root@ubuntu:~/auditor# gcc -Wall -o auditor file_permission_auditor.c
root@ubuntu:~/auditor# ./auditor

===== FILE PERMISSION AUDITOR =====
 1. Scan a directory
 2. View unsafe files
 3. Fix a selected file
 4. Fix all unsafe files
 5. Permission report
 6. View audit log
 0. Exit
Choice:
```

CHAPTER – 6

IMPLEMENTATION

6.1 Module-wise Implementation

All the code is in file_permission_auditor.c. It has the data structure for one scanned file (Entry), the scanner, the risk classification, the display functions, the audit log, the permission correction and the main menu. The scanned files are kept in an array that grows with realloc().

6.2 Algorithm / Pseudocode

1. Show the menu and read the choice.
2. For Scan: read the directory and check it with stat().
3. Open the directory, and for every entry get its details with lstat().
4. Skip symbolic links. Classify the entry and update the counters.
5. If the entry is a directory, repeat from step 3 for it.
6. For Fix: remove SetUID, SetGID, group-write and other-write (and other-execute on files).
7. Call chmod() and add a line to audit.log.
8. Repeat until the user chooses Exit.

6.3 Core C Implementation

Risk classification:

```
/* Risk analysis rules */
static Risk classify(mode_t m, int is_dir, char *reason, size_t n)
{
    if (m & S_ISUID)  { snprintf(reason, n, "SetUID bit");          return CRITICAL; }
    if (m & S_ISGID && !is_dir) { snprintf(reason, n, "SetGID bit"); return CRITICAL; }
    if ((m & S_IWOTH) && !(is_dir && (m & S_ISVTX)))
                      { snprintf(reason, n, "other write"); return CRITICAL; }
    if (m & S_IWGRP)  { snprintf(reason, n, "group write");       return WARNING;  }
    if (!is_dir && (m & S_IXOTH))
                      { snprintf(reason, n, "other exec"); return WARNING; }
    snprintf(reason, n, "restricted");
    return SAFE;
}
```

Recursive scan:

```
static void scan_dir(const char *dir)
{
    DIR *d = opendir(dir);
    if (!d) { fprintf(stderr, "  [skip] %s: %s\n", dir, strerror(errno)); return; }

    struct dirent *de;
    while ((de = readdir(d)) != NULL) {
        if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, "..")) continue;
        char path[PATH_MAX];
        snprintf(path, sizeof(path), "%s/%s", dir, de->d_name);

        struct stat st;
        if (lstat(path, &st) == -1) { perror(path); continue; }
        if (S_ISLNK(st.st_mode)) continue;          /* symlink permissions are not meaningful */
        add_entry(path, &st);
        if (S_ISDIR(st.st_mode)) scan_dir(path);    /* recursion */
    }
    closedir(d);
}
```

Fixing a file and writing the log:

```
static void write_log(const char *path, mode_t oldm, mode_t newm)
{
    FILE *f = fopen(LOG_FILE, "a");
    if (!f) { perror("audit.log"); return; }
    char ts[32];  time_t now = time(NULL);
    strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S", localtime(&now));
    fprintf(f, "%s | %s | old=%04o new=%04o\n", ts, path, oldm & 07777, newm & 07777);
    fclose(f);
}

static int fix_entry(Entry *e)
{
    mode_t newm = e->mode & ~(S_ISUID | S_ISGID | S_IWGRP | S_IWOTH);
    if (!e->is_dir) newm &= ~S_IXOTH;
    if (e->is_dir && (e->mode & S_IWOTH)) newm |= S_ISVTX;   /* keep shared dirs sticky */
    if (newm == e->mode) return 0;
    if (chmod(e->path, newm & 07777) == -1) { perror(e->path); return -1; }
    write_log(e->path, e->mode, newm);
    printf("  Fixed %s : %04o -> %04o\n", e->path, e->mode & 07777, newm & 07777);

    if (e->risk == CRITICAL) n_crit--; else if (e->risk == WARNING) n_warn--;
    n_safe++;
    e->mode = newm;  e->risk = SAFE;
    snprintf(e->reason, sizeof(e->reason), "restricted");
    return 1;
}
```

6.4 Linux System Calls / APIs Used

API / Function	Purpose
stat()	Checks that the input is a directory.
lstat()	Gets mode, owner and group of each entry.
opendir() / readdir() / closedir()	Reads directory contents.
chmod()	Changes the permission bits.
fopen() / fprintf() / fclose()	Writes the audit log.
time() / localtime() / strftime()	Time of change.
realloc() / free()	Stores the scan results.

6.5 Process / Thread Implementation

The tool runs as one process with one thread. No child process is created with fork(). The recursion in scan_dir() keeps track of the directories that are open at any moment.

6.6 IPC / Synchronization Implementation

Since there is only one process, no inter-process communication is needed. The only shared resource is audit.log, which is opened in append mode, written and closed for every change.

6.7 Memory Management Implementation

The list of scanned files starts small and its size is doubled with realloc() when it is full. It is freed before a new scan and when the program exits. Strings are copied with snprintf() and strncpy() using fixed lengths, and the return values of opendir(), lstat(), chmod() and realloc() are checked. valgrind reported no errors and no leaks for a test run.

6.8 Optional User Interface Implementation

The interface is a simple numbered menu in the terminal: scan a directory, view unsafe files, fix a selected file, fix all unsafe files, permission report, audit log and exit.

CHAPTER – 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
gcc -Wall -Wextra -o auditor file_permission_auditor.c
```

7.2 Execution Commands

```
./auditor
```

A test folder named test was created with files of different permissions before running the tool.

7.3 Sample Inputs and Outputs

File	Permission	Expected result
test/config.cfg	600	Safe
test/docs/notes.txt	644	Safe
test/docs/report.txt	666	Critical (other write)
test/scripts/deploy.sh	777	Critical (other write)
test/scripts/backup.sh	755	Warning (other exec)
test/shared/public.txt	664	Warning (group write)
test/setuid_tool	4755	Critical (SetUID)
3 directories	755	Safe

Test Case	Input	Observed Result
Scan a directory	1 → test	10 items examined
Permission report	5	Safe 5, Warning 2, Critical 3
View unsafe files	2	5 files listed with risk and reason
Fix one file	3 → 5	backup.sh changed from 0755 to 0754
Fix all	4	4 files corrected
Report after fixing	5	Safe 10, Warning 0, Critical 0
View audit log	6	5 records with old and new permission
Fix before scanning	3	Run a scan first.
Wrong menu number	9	Invalid choice.
Wrong directory	1 → nodir	Invalid directory: nodir

7.4 Screenshots of Execution

```
root@ubuntu:~/auditor# gcc -Wall -o auditor file_permission_auditor.c
root@ubuntu:~/auditor# ./auditor

===== FILE PERMISSION AUDITOR =====
 1. Scan a directory
 2. View unsafe files
 3. Fix a selected file
 4. Fix all unsafe files
 5. Permission report
 6. View audit log
 0. Exit
Choice:
```

```
Choice: 1
Enter directory to scan: test
  Scan complete. 10 item(s) examined.

Choice: 5

========== PERMISSION REPORT ==========
 Directory scanned : test
 Total scanned     : 10
 Safe              : 5
 Warning           : 2
 Critical          : 3
=======================================
```

```
Choice: 2

ID  PERMISSION  OCTAL  RISK      REASON       FILE
-----------------------------------------------------------------
1   -rw-rw-r--  664    WARNING   group write  test/shared/public.txt
2   -rwsr-xr-x  4755   CRITICAL  SetUID bit   test/setuid_tool
4   -rwxrwxrwx  777    CRITICAL  other write  test/scripts/deploy.sh
5   -rwxr-xr-x  755    WARNING   other exec   test/scripts/backup.sh
8   -rw-rw-rw-  666    CRITICAL  other write  test/docs/report.txt
```

```
root@ubuntu:~/auditor# ls -lR test
test:
total 16
-rw------- 1 root root    2 Oct 31 01:49 config.cfg
drwxr-xr-x 2 root root 4096 Oct 31 01:49 docs
drwxr-xr-x 2 root root 4096 Oct 31 01:49 scripts
-rwsr-xr-x 1 root root    0 Oct 31 01:49 setuid_tool
drwxr-xr-x 2 root root 4096 Oct 31 01:49 shared

test/docs:
total 8
-rw-r--r-- 1 root root 2 Oct 31 01:49 notes.txt
-rw-rw-rw- 1 root root 2 Oct 31 01:49 report.txt

test/scripts:
total 8
-rwxr-xr-x 1 root root 2 Oct 31 01:49 backup.sh
-rwxrwxrwx 1 root root 2 Oct 31 01:49 deploy.sh

test/shared:
total 4
-rw-rw-r-- 1 root root 2 Oct 31 01:49 public.txt
```

```
Choice: 3
Enter file ID to fix (see 'View issues'): 5
  Fixed test/scripts/backup.sh : 0755 -> 0754

Choice: 4
  Fixed test/shared/public.txt : 0664 -> 0644
  Fixed test/setuid_tool : 4755 -> 0754
  Fixed test/scripts/deploy.sh : 0777 -> 0754
  Fixed test/docs/report.txt : 0666 -> 0644
  4 item(s) corrected.
```

```
Choice: 5

========== PERMISSION REPORT ==========
 Directory scanned : test
 Total scanned     : 10
 Safe              : 10
 Warning           : 0
 Critical          : 0
=======================================
```

```
root@ubuntu:~/auditor# ls -lR test
test:
total 16
-rw------- 1 root root    2 Oct 31 01:49 config.cfg
drwxr-xr-x 2 root root 4096 Oct 31 01:49 docs
drwxr-xr-x 2 root root 4096 Oct 31 01:49 scripts
-rwxr-xr-- 1 root root    0 Oct 31 01:49 setuid_tool
drwxr-xr-x 2 root root 4096 Oct 31 01:49 shared

test/docs:
total 8
-rw-r--r-- 1 root root 2 Oct 31 01:49 notes.txt
-rw-r--r-- 1 root root 2 Oct 31 01:49 report.txt

test/scripts:
total 8
-rwxr-xr-- 1 root root 2 Oct 31 01:49 backup.sh
-rwxr-xr-- 1 root root 2 Oct 31 01:49 deploy.sh

test/shared:
total 4
-rw-r--r-- 1 root root 2 Oct 31 01:49 public.txt
```

```
root@ubuntu:~/auditor# cat audit.log
2026-10-31 01:49:47 | test/scripts/backup.sh | old=0755 new=0754
2026-10-31 01:49:47 | test/shared/public.txt | old=0664 new=0644
2026-10-31 01:49:47 | test/setuid_tool | old=4755 new=0754
2026-10-31 01:49:47 | test/scripts/deploy.sh | old=0777 new=0754
2026-10-31 01:49:47 | test/docs/report.txt | old=0666 new=0644
```

```
Choice: 3
  Run a scan first.

Choice: 9
  Invalid choice.

Choice: 1
Enter directory to scan: nodir
  Invalid directory: nodir
```

7.5 Demonstration of OS Concepts

- File system: files and directories are accessed through the Linux file system.
- File permissions: owner, group and other read/write/execute bits are checked.
- Ownership: the owner and group of each file are read from the inode.
- Directory traversal: sub-directories are visited recursively.
- System calls: stat, lstat, opendir, readdir and chmod talk to the kernel.
- Access control: unsafe access is identified and removed.
- File handling: the audit log is written and read as a file.

CHAPTER – 8

CONCLUSION

8.1 Summary of the Project

The File Permission Auditor gives a simple way to check and understand file and directory permissions on Linux. It scans the selected directory, checks the permissions of the owner, group and others and points out unsafe settings. It can also correct them and keeps a log of what was changed. The project shows basic operating system ideas such as file permissions, ownership, access control, file systems and system calls.

8.2 Objectives Achieved

- A Linux-based permission checker was developed in C.
- Directories are scanned recursively.
- Files are classified as Safe, Warning or Critical.
- Unsafe permissions can be corrected for one file or all files.
- Changes are saved in an audit log.
- A report with the totals is generated.

8.3 Results Achieved

The program was compiled with no warnings and tested on a folder of 10 items. It found 5 Safe, 2 Warning and 3 Critical items, as expected. After fixing, all 10 items were Safe, and the audit log had 5 records with the old and new permissions.

8.4 Skills and Learning Outcomes

- Using Linux system calls from C.
- Understanding permission bits, ownership and the SetUID/SetGID bits.
- Recursive programming and dynamic memory.
- File handling and time functions.
- Testing with prepared test cases, and using Git and GitHub.

CHAPTER – 9
REFERENCES

9.1 Books

- Silberschatz, Galvin, Gagne – Operating System Concepts, 10th Edition, Wiley.
- Michael Kerrisk – The Linux Programming Interface, No Starch Press.
- Stevens and Rago – Advanced Programming in the UNIX Environment, 3rd Edition, Addison-Wesley.
- Kernighan and Ritchie – The C Programming Language, 2nd Edition.

9.2 Research Papers

- J. H. Saltzer and M. D. Schroeder – The Protection of Information in Computer Systems, Proceedings of the IEEE, 1975.
- D. M. Ritchie and K. Thompson – The UNIX Time-Sharing System, Communications of the ACM, 1974.

9.3 Linux / GNU Documentation

- Linux manual pages: stat(2), chmod(2), opendir(3), readdir(3), inode(7) – man7.org
- GNU C Library manual and GCC documentation – gnu.org

9.4 Online Resources

- Project repository: github.com/mynamesteenzz-droid/KLH-CSIT-2025-29-22-File-Permission-Auditor
- Course notes and laboratory material.